SPRINT 3 · PARTE 2

Dívida técnica e refatoração

Projeto P07 - Controle de Frequência · Branch sprint3/testes · Relatório baseado nos commits reais

1. Escopo, identificação e priorização

O diagnóstico examinou o serviço de chamada por código, DTOs e componentes React relacionados, procurando coesão, acoplamento, responsabilidades (SRP/DIP) e code smells. Foram registrados **oito itens** em `docs/qualidade/divida-tecnica.md`. Os itens **DT-02, DT-04 e DT-01** foram escolhidos por afetarem diretamente o serviço testado pela suíte v2 e poderem ser corrigidos por extrações pequenas, em commits separados.

A leitura foi manual com apoio do ChatGPT. Não foram executadas ferramentas específicas de análise estática como jscpd, SonarQube ou madge. Uma sugestão de LLM não é considerada confirmação automática: as duplicações e concentrações de responsabilidades foram verificadas contra trechos do repositório.

2. DT-02 — SQL de bloqueio duplicado

Antes: a consulta PostgreSQL `SELECT "id" FROM "aulas" ... FOR UPDATE` aparecia nos métodos `abrir`, `encerrar` e `registrar`. **Depois:** `bloquearAula(tx, aulaId)` encapsulou a operação. **Técnica:** Extract Function. **Resultado arquitetural:** reduziu código duplicado e o risco de uma alteração de sincronização ser aplicada de maneira inconsistente. O bloqueio da linha da aula foi preservado.

Commit: aca6e77. **Testes posteriores:** 48/48 no arquivo `dt02-testes.txt`. **Build posterior:** `nest build` sem erro registrado. Não existe log específico de execução pré-DT02 salvo no repositório; a cobertura da v2 medida antes das refatorações constitui uma referência estrutural, mas não substitui esse log.

3. DT-04 — Leitura do tempo encapsulada

Antes: leituras de `Date.now()` e `new Date()` dispersas nos fluxos de abertura, estado e encerramento. **Depois:** o método `agora()` concentra a obtenção do instante. **Técnica:** Extract Function / Encapsulate Clock. **Resultado arquitetural:** um ponto único para manutenção do critério de expiração e tempo simulado em testes.

Commit: 95779b6. **Testes v2:** antes 48/48, depois 48/48 (`dt04-antes-testes.txt`, `dt04-depois-testes.txt`). Builds posteriores e anteriores com invocação `nest build` e sem erros nos logs.

4. DT-01 — Separação do registro de presença

Antes: `registrar` acumulava verificação de matrícula, transação, tentativas e tradução dos resultados em erros HTTP. **Depois:** `buscarMatricula` separa verificação de acesso; `interpretarRegistro` concentra a tradução de estados da transação. **Técnica:** Extract Method. **Resultado arquitetural:** melhora a legibilidade e a divisão interna de responsabilidades, mantendo a sequência de validações.

Commit: e5136f6. **Testes v2:** antes 48/48, depois 48/48 (`dt01-antes-testes.txt`, `dt01-depois-testes.txt`). Os logs de build não registram erros.

Dívida	Commit	v2 antes	v2 depois	Build depois
DT-02	aca6e77	Sem log específico	48/48	Sem erro no log
DT-04	95779b6	48/48	48/48	Sem erro no log
DT-01	e5136f6	48/48	48/48	Sem erro no log

5. Comparação dos métodos de identificação

A análise manual auxiliada por IA identificou não apenas repetição textual do `FOR UPDATE`, mas também o motivo de preservá-lo: impedir corridas ao reabrir ou registrar uma chamada. Uma análise estática poderia detectar duplicação de trechos e funções extensas, mas não justificaria isoladamente o uso transacional. O diagnóstico de acoplamento ao relógio exigiu interpretar a regra de expiração, enquanto a divisão do método `registrar` decorreu de seus múltiplos motivos para mudar. Não se atribuem resultados a ferramentas não executadas.

6. Riscos restantes e conclusão

• Os cinco itens restantes permanecem abertos: DT-03, DT-05, DT-06, DT-07 e DT-08, conforme inventário atualizado.

• Os 48 testes unitários da v2 passaram após as três refatorações; isso sustenta preservação dos comportamentos testados, mas não prova equivalência universal do sistema.

• As verificações usam mocks do Prisma. Falta teste de integração com PostgreSQL para concorrência, rollback e confirmação da eficácia real de `FOR UPDATE`.

• Os logs de build registram `nest build` sem erro. A persistência do código de saída dos builds é parcial; não se deve inventar evidência que não foi arquivada.

Os três commits de código foram preservados separadamente na branch. A equipe deve confirmar o inventário atualizado, revisar os riscos e aprovar um pull request antes de integrar à `main`.

Fontes verificáveis

Inventário: `docs/qualidade/divida-tecnica.md`. Evidências: `docs/qualidade/evidencias/dt02-testes.txt`, `dt04-{antes,depois}-testes.txt` e `dt01-{antes,depois}-testes.txt`, além dos logs de build. Commits: `aca6e77`, `95779b6` e `e5136f6` no repositório `leolqmr/trabalho-2.0`.

Documento consolidado em 08/11/2026 com base no GitHub. Revisar e aprovar com a equipe antes da entrega.